微电路设计小组报告

学院 国家卓越工程师学院

班级 明月科创实验班 1 班

学号 20225188

姓名 洪竞权，王令凯, 贺禄文

2026 年 1 月 17 日

目录

1 项目概述 **4**

1.1 核心目标 4

1.2 核心成果 4

2 硬件电路设计 **6**

2.1 核心架构设计 6

2.2 整体原理图与布局 6

2.3 关键模块设计 7

2.4 迭代优化 9

3 嵌入式软件控制系统 **9**

3.1 控制核心与硬件适配 9

3.1.1 核心芯片与 HAL 驱动库机制(STM32F407ZGT6 + STM32F4x_HAL_Driver) 9

3.1.2 外设适配：电机、传感器、蓝牙模块、语音模块、树莓派 10

3.2 软件架构设计 17

3.3 核心技术实现 18

3.3.1 运动控制算法：串级 PID 角度式闭环控制 18

3.3.2 数据采集：IMU 角度检测与编码器测速（mm/s） 18

3.3.3 控制效果：直线抗干扰与转弯角度精准控制 19

3.3.4 通信协议开发：Web 端 3D 智能地面站（V4.7） 19

4 上位机与系统协同实现 **20**

4.1 ROS 2 系统协同架构 20

4.2 ROS 2 软件工程组织与功能划分 21

4.3 导航与路径规划实现 22

4.3.1 RViz 导航交互与路径可视化 22

4.3.2 关键导航参数设置 23

4.4 导航执行状态判定与流程控制 23

4.5 系统协同流程总结 24

5 功能验证与测试结果 **24**

5.1 基础功能测试 24

5.1.1 硬件功能测试 24

5.1.2 数据采集功能测试 25

5.1.3 充电功能测试 25

5.2 核心功能测试 25

5.2.1 运动控制测试 25
5.2.2 安全防护测试 25
5.2.3 导航功能测试 25
5.3 交互功能测试 25
5.3.1 遥控功能测试 25
5.3.2 语音交互测试 26
5.3.3 网页交互与数据可视化测试 26

6 项目过程管理 **26**
6.1 项目进度 26
6.2 分工协作 26

7 项目总结 **26**
7.1 核心创新点 26
7.2 经验与教训 27

1 项目概述

本项目面向室内扫地机器人平台，完成硬件电路集成、嵌入式控制系统构建与系统协同联调。总体架构采用“下位机（STM32）实时执行控制 + 树莓派串口桥接 + 上位机（PC）算法规划”的分布式模式：下位机负责电机闭环、传感器采集与外设控制；树莓派负责与下位机串口通信，并作为 ROS 2 网络中的边缘节点转发数据；上位机负责 SLAM 建图、路径规划与可视化展示，并将控制指令经 ROS 2 下发至树莓派，再由树莓派转发到下位机执行。

1.1 核心目标

项目目标包括五个层面：完成扫地机器人硬件电路的集成设计与制作；开发嵌入式软件控制系统，实现电机、传感器、外设协同控制；部署智能导航算法以支持自主建图、路径规划、避障与回充；实现多元交互方式以提升操控便利性；完成全功能测试验证，确保系统稳定可靠运行。

1.2 核心成果

项目成果覆盖硬件、软件与算法三条主线并形成闭环落地。硬件方面完成基于 STM32F407ZGT6 的电路集成设计，输出完整原理图、PCB 布线与 3D 设计文件；软件方面搭建分层架构的嵌入式控制系统并适配 FreeRTOS，实现多任务调度与外设协同；算法方面部署基于 ROS 2（DDS 通信机制）的导航系统，支持仿真与实机无缝复用，并完成跨设备协同联调。系统功能按“基础功能/升级功能”划分汇总如表 1。

表 1: 功能范围（基础与升级）

类别	功能说明
基础功能	电机控制（底盘轮、边刷启停与转速调节）；清洁设备控制（刷子、吸尘器、水泵）；数据采集（编码器速度、红外、光电门）；电源管理（14.8V→5V、5V→3.3V、19.8V 充电/SLM6900）；安全防护（左右/前进悬空红外防跌落、碰撞光电门）；基础通信（UART 与树莓派/语音/蓝牙模块通信）；辅助功能（LED、蜂鸣器、按键交互）。
升级功能	运动控制优化（串级 PID 角度式闭环，直线抗干扰与转向精度提升）；多元交互升级（APP、语音含 AI 对话、键鼠遥控与数据可视化）；通信协议创新（自研 Web Bluetooth，Service UUID：0xFFE0，浏览器本地/Github Page 网页控制）；智能导航拓展（混合路径规划：轨迹跟随、“弓”字形覆盖、直线返航；动态避障；自主回充导航）；系统协同增强（ROS 2 分布式通信：树莓派雷达采集、上位机 SLAM/规划、下位机执行反馈）；可视化监控（RViz 显示位姿/路径/轨迹，支持手动指定导航目标点）；控制精度提升（融合 IMU 角度与编码器测速的电机闭环控制）。

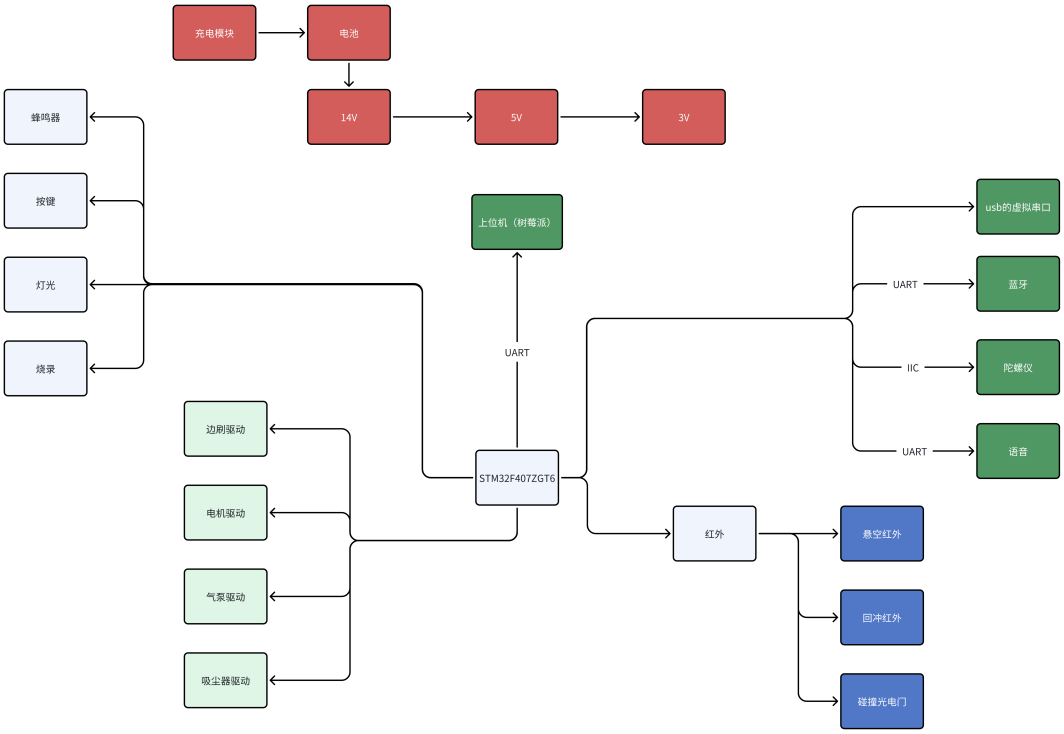


图 1: 系统逻辑框图

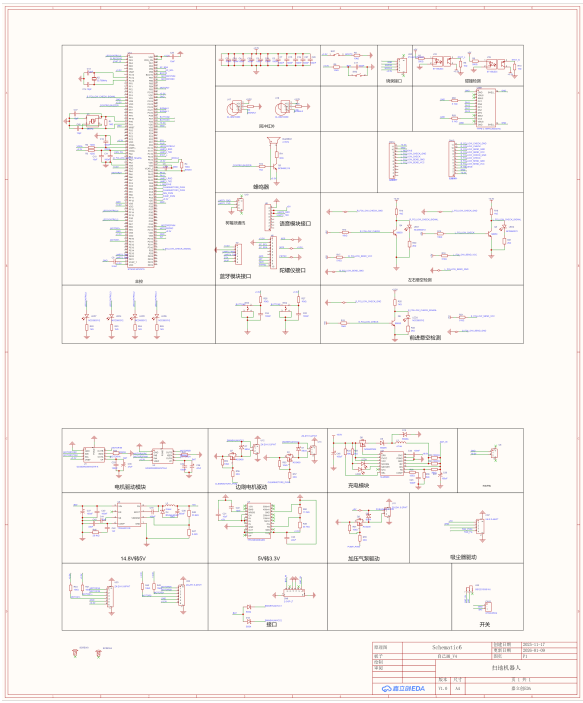
2 硬件电路设计

2.1 核心架构设计

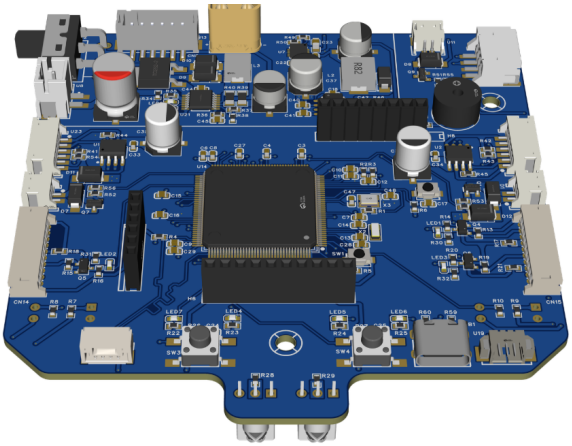
硬件系统以 STM32F407ZGT6 作为核心控制单元，围绕电源与充电管理、外设接口、传感模块、驱动模块与辅助交互模块构建集成电路板。设计中对强电/弱电进行分区布线，电源域进行清晰划分（14.8V 电池域、5V 外设域、3.3V 数字域），并通过滤波与保护器件降低瞬态电流与开关噪声对控制与传感的影响，从而为后续 FreeRTOS 多任务与高频控制环提供稳定硬件基础。

2.2 整体原理图与布局

整板原理图、PCB 布线与 3D 设计文件已完成输出。课程报告中建议分别插入整体原理图与 PCB 3D 视图，以便直观展示模块分区与接口布局，如图 ?? 与图 ??。



（a）整体原理图



（b）PCB 3D 视图

图 2: 硬件系统整体设计展示

2.3 关键模块设计

为保证描述完整且减少条目式表达，本节将关键模块信息归纳于表 2，并在后续文字中对关键器件与接口用途进行说明。

表 2: 硬件关键模块设计要点

模块	关键器件/接口	设计要点（与原文信息一致）
主控模块	R26/R27=10kΩ; C24/C25=100nF; 烧录/电源/外设接口; LED4/LED5/LED7; W3/BUTTON2/SW4	去耦与上拉保证 MCU 稳定；接口预留便于调试与扩展；指示灯与按键用于状态提示与交互。
电源模块	14.8V→5V; 5V→3.3V; SLM6900(19.8V 充电); 4.7uH 电感; 100uF 电容（C39/C44）; 100nF 滤波; B340A; NCE60P50K	多电压域供电；储能/滤波抑制纹波；保护器件提升可靠性与安全性。
外设接口模块	UART1/UART3/UART4 I2C1; 树莓派/语音/MPU9250 接口; HC-06（含 BC84215）; Type-C 16PIN（2MD(073)）	多串口支撑多源控制；I2C 支撑 IMU；Type-C 供电与维护；蓝牙模块适配保证电平与连接可靠。
传感模块	红外（左右/前进悬空、回充 XL-IRM1838D）；碰撞光电门（IFHIT_R、XY-CH2 2000）; R10=1kΩ; ST150(ZD); 3.3V/5V 供电域	防跌落/碰撞为安全关键；供电域区分减少互扰；接口与电阻器件保证输入稳定。
驱动模块	底盘轮/边刷电机驱动；加压气泵驱动；吸尘器驱动；CLE 控制接口；编码器信号接口	强电驱动与控制信号隔离；接口规范化支撑闭环控制与调试。
辅助功能模块	蜂鸣器（CONTROL-BUZZER、BUZZER1）; LED 灯光	声光提示用于报警与状态显示，提升可维护性。

2.4 迭代优化

硬件初版在走线与功能扩展方面进行了问题检查，主要包括走线不够优化、后续功能增加带来的接口与布局调整需求。针对这些问题，迭代方向以优化布局为主：通过强弱电分区更清晰、关键回流路径更短、功率走线加粗、增加测试点与指示电路等方式提升板级稳定性与调试效率。此处建议在报告中补充“迭代前后局部对比图”展示优化结果。

3 嵌入式软件控制系统

3.1 控制核心与硬件适配

3.1.1 核心芯片与 HAL 驱动库机制(STM32F407ZGT6 + STM32F4x_HAL_Driver)

本项目下位机控制平台为 STM32F407ZGT6。该芯片采用 ARM Cortex-M4 32 位 RISC 内核，最高主频 168MHz，集成 FPU 与 DSP 指令集，使得姿态角（float）处理与控制算法计算更加高效，从而避免浮点运算占用过多 CPU 资源。芯片提供 1MB 程序存储空间，可满足移植 FreeRTOS、LwIP 以及复杂控制逻辑的代码规模需求，同时也便于划分参数存储区域。实际调试中系统任务数量达到约 9 项，在未进行额外内存优化的情况下仍可稳定运行并完成控制闭环与多外设协同。

图 3: STM32F407ZGT6 与开发环境示意（占位图）

软件驱动库采用 ST 官方主推的 STM32F4x_HAL_Driver。HAL 库相比旧标准库更强调抽象与复用，其关键机制体现在三方面。第一，MSP（MCU Support Package）机制实现“通用配置与板级配置分离”，例如串口初始化时 `HAL_UART_Init()` 负责波特率、

停止位、校验位等通用参数，而 HAL_UART_MspInit() 负责 GPIO 复用映射、时钟使能与 NVIC 优先级等板级配置。第二，Callback 机制对中断处理进行标准化：硬件触发中断后进入 USARTx_IRQHandler()，随后调用 HAL_UART_IRQHandler(&huart) 完成标志位清除与中断类型判断，再调用弱定义回调函数 HAL_UART_RxCpltCallback() 承载业务逻辑，从而避免在 ISR 中直接堆叠业务代码。第三，HAL 对 DMA 进行了默认集成，例如 HAL_UART_Receive_DMA() 可快速启用 DMA 接收，降低 CPU 占用并提升吞吐能力。开发工具链采用 Keil + STM32CubeMX 完成工程生成、外设配置与调试联动。

3.1.2 外设适配：电机、传感器、蓝牙模块、语音模块、树莓派

本节对外设适配按照“电机驱动与闭环控制、传感器采集、蓝牙/语音交互、树莓派串口协议”进行展开。整体系统架构属于典型闭环控制系统，按层划分为底层驱动层（PWM 生成等）、算法层（PID 控制、里程计/姿态融合）与应用调度层（FreeRTOS 任务）。

(1)电机适配：Drivers Layer + Feedback Layer + Algorithm Layer + Application Layer 电机适配覆盖驱动轮与功能电机两类。驱动轮采用 TIM4 产生 PWM 波形，并通过双通道互补逻辑实现方向与速度控制：左轮使用 TIM_CHANNEL_1/2，右轮使用 TIM_CHANNEL_3/4；当速度为正时通道 A 输出 PWM、通道 B 为 0，对应正转；当速度为负时通道 A 为 0、通道 B 输出 PWM，对应反转。功能电机包括水泵、清扫刷与吸尘风机：水泵使用 TIM3 通道 1（SetPumpSpeed），清扫刷使用 TIM3 通道 3/4（SetClearMotorSpeed），吸尘风机使用 TIM8 通道 2（SetPumpingSpeed），从而实现水流量、刷速与吸力的 PWM 调速。

```
void SetWheelSpeed(int16_t speed_l, int16_t speed_r)
{
    // 左轮 TIM4 CH1/CH2
    if(speed_l > 0){
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, speed_l);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, 0);
    }else{
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, 0);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, -speed_l);
    }

    // 右轮 TIM4 CH3/CH4
    if(speed_r > 0){
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, speed_r);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, 0);
    }else{
```

```
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, -speed_r);
    }
}
```

Code Listing 1: MotorWheel.c：驱动轮速度设置（方向与 PWM 互补逻辑）

为实现 PID 闭环控制，需要反馈当前实际速度。编码器采集未使用 STM32 硬件 Encoder Mode，而采用 GPIO 轮询检测（Software Polling）：左轮引脚为 GPIOE_PIN_11，右轮引脚为 GPIOB_PIN_3。在 FreeRTOS 的 CountTask（myTask04）中死循环检测引脚电平变化，捕获跳变（0→1 或 1→0）后累加计数器 `cnt_l/cnt_r`。在 SpeedTask（myTask05）中每 10ms（`osDelay(10)`）读取计数并乘以系数 `SpeedArea` 得到速度 `sp_l/sp_r`，随后清零计数器。

```
volatile int32_t cnt_l = 0, cnt_r = 0;
volatile float sp_l = 0.0f, sp_r = 0.0f;
const float SpeedArea = 100.0f; // 10ms计数换算系数

void CountTask(void const *argument)
{
    uint8_t last_l = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_11);
    uint8_t last_r = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_3);

    for(;;){
        uint8_t now_l = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_11);
        uint8_t now_r = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_3);

        if(now_l != last_l){ cnt_l++; last_l = now_l; }
        if(now_r != last_r){ cnt_r++; last_r = now_r; }
    }
}

void SpeedTask(void const *argument)
{
    for(;;){
        osDelay(10);
        sp_l = cnt_l * SpeedArea;
        sp_r = cnt_r * SpeedArea;
        cnt_l = 0; cnt_r = 0;
    }
}
```

Code Listing 2: freertos.c：编码器轮询计数与 10ms 速度估计（关键变量保留）

PID 控制算法主要在 freertos.c 中实现（而不是完全调用 PIDController.c 的通用库）。控制流程为：计算误差 `error = target_speed - current_speed`；更新积分项 `integral += error` 并对积分进行限幅（防积分饱和，阈值约 100000）；计算微分项 `derivative = error - last_error`；输出限幅（`outputmax=350` 防止 PWM 溢出）；最后通过 `PWMGeneration_l/r` 写入定时器寄存器完成驱动。

```
float PIDSpeedLoop(float target_speed, float current_speed,
                   float *integral, float *last_error)
{
    const float Kp = 0.8f, Ki = 0.02f, Kd = 0.1f;
    const float outputmax = 350.0f;

    float error = target_speed - current_speed;
    *integral += error;
    if(*integral > 100000.0f)  *integral = 100000.0f;
    if(*integral < -100000.0f) *integral = -100000.0f;

    float derivative = error - *last_error;
    *last_error = error;

    float out = Kp*error + Ki*(*integral) + Kd*derivative;
    if(out > outputmax)  out = outputmax;
    if(out < -outputmax) out = -outputmax;
    return out;
}
```

Code Listing 3: freertos.c：PID 速度环（积分限幅与输出限幅，outputmax=350）

如表 3 所示，FreeRTOS 通过多个并行任务对编码器采集、速度计算、底盘控制、功能电机控制以及里程计数据回传等功能进行统一调度，实现了系统功能的并行运行。

表 3: FreeRTOS 任务划分与功能说明

任务名称	对应函数	优先级	功能描述
myTask04	CountTask	Normal	编码器计数任务。持续轮询 GPIO 电平变化并累加编码器脉冲计数。当转速较高时，软件轮询方式可能存在丢步风险，后续可优化为定时器编码器模式或外部中断方式。
myTask05	SpeedTask	Normal	速度计算任务。每 10ms 运行一次，将编码器脉冲数转换为左右轮速度 sp_l/sp_r，作为 PID 闭环控制的反馈输入。
myTask06	pidcontrol	Normal	底盘闭环控制核心任务。接收控制指令（如串口数据 `uart4data`），根据目标速度计算 PID 输出，并持续更新电机 PWM 占空比，实现稳定速度控制。
myTask08	SenfSerialTask	Normal	高级运动控制任务。解析串口控制指令，实现 `TurnToAngle_90`（原地旋转 90°）、`forward_1`（直线行驶）等复合动作，并在内部调用速度 PID 进行精细调节。
myTask03	GimbalTask	Normal	功能电机控制任务。根据接收到的控制命令（如 `Usart_Receive`），控制水泵、清扫刷以及吸尘风机的启停与运行状态。
myTask07	GoChassisTask	Normal	里程计计算与数据上传任务。基于左右轮速度与 IMU 航向角计算机器人位姿 (x, y, yaw)，并通过 `send_wheel_data` 接口以固定周期发送给上位机或树莓派。

(2) 传感器适配：IMU (I2C) + GPIO (红外/碰撞) 传感器系统采用两种接入方式：IMU 通过 I2C 总线通信，红外/碰撞传感器通过 GPIO 数字电平读取，并在 FreeRTOS 的不同任务中调度。IMU 采用 MPU9250，内部集成 AK8963 磁力计（16bit 模式下灵敏度约 0.15μT/LSB），是纠正 Yaw 航向角漂移的重要来源。姿态解算利用 MPU9250 内部 DMP（Digital Motion Processor）输出欧拉角（Yaw/Pitch/Roll），从而减轻 STM32 的运算负担。

IMU 通信使用 STM32 硬件 I2C（`hi2c1`），MPU9250 地址为 `0x68`，磁力计 AK8963 地址为 `0x0C`。初始化中开启 Bypass 模式（例如 `MPU_INTBP_CFG_REG, 0x82`），使 STM32 可

直接访问 AK8963。姿态解算不手写互补滤波或卡尔曼滤波，而调用 mpu_dmp_init() 与 mpu_mpl_get_data(&pitch,&roll,&yaw) 获取姿态角。原文指出 mpu9250_Start() 内部包含 while(1) 死循环，意味着 IMU 任务启动后持续更新全局变量 pitch/roll/yaw。

```
volatile float pitch=0.0f, roll=0.0f, yaw=0.0f;

void StartDefaultTask(void const * argument)
{
    mpu_dmp_init();
    for(;;){
        mpu_mpl_get_data(&pitch, &roll, &yaw);
        osDelay(5);
    }
}
```

Code Listing 4: mpu9250：DMP 姿态解算任务（持续更新 pitch/roll/yaw）

GPIO 传感器适配则通过读取数字电平判断状态。红外跟随/避障传感器由 FollowCheck() 读取 SFollowCheck/LFollowCheck/RFollowCheck 引脚，结果存入全局变量 SF/LF/RF；在控制任务中根据 SF/LF/RF 执行避障或跟随逻辑（例如当前方 SF==0 时后退或停止）。悬崖/跌落传感器由 ILight() 读取 GPIOE_PIN_4 与 GPIOA_PIN_15，更新 IR/IL 变量用于台阶防跌落。碰撞开关由 CollReceive() 读取 L_Rev 作为最后安全保障。执行器方面，PullBuzzer() 控制蜂鸣器，TurnOnLED() 控制 LED 指示系统状态。

```
volatile uint8_t SF=1, LF=1, RF=1;
volatile uint8_t IL=1, IR=1;
volatile uint8_t L_Rev=1;

void FollowCheck(void){
    SF = HAL_GPIO_ReadPin(SFollowCheck_GPIO_Port, SFollowCheck_Pin);
    LF = HAL_GPIO_ReadPin(LFollowCheck_GPIO_Port, LFollowCheck_Pin);
    RF = HAL_GPIO_ReadPin(RFollowCheck_GPIO_Port, RFollowCheck_Pin);
}

void ILight(void){
    IR = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_4);
    IL = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_15);
}

void CollReceive(void){
    L_Rev = HAL_GPIO_ReadPin(L_Rev_GPIO_Port, L_Rev_Pin);
}
```

```c

void PullBuzzer(uint8_t on){
    HAL_GPIO_WritePin(BUZZ_GPIO_Port, BUZZ_Pin, on ? GPIO_PIN_SET :
        GPIO_PIN_RESET);
}

void TurnOnLED(uint8_t on){
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, on ? GPIO_PIN_SET :
        GPIO_PIN_RESET);
}
```

Code Listing 5: Sensors.c：红外/跌落/碰撞采集与蜂鸣器 LED 控制（关键变量 SF/LF/RF/IL/IR/L_Rev）

(3) 蓝牙模块适配：中断接收 + 全局变量共享 + 任务轮询 蓝牙适配采用经典架构：蓝牙模块通过 UART3/UART4 接收手机/上位机发送的数据，触发串口接收中断；中断服务读取数据寄存器并更新全局变量 `Usart_Receive`；FreeRTOS 任务轮询该变量并执行动作。控制逻辑分散在两个任务：GimbalTask 负责清洁功能（边刷、吸尘、水泵），SenfSerialTask 负责底盘运动（包含蓝牙遥控与基于 `SF/LF/RF` 的简单避障）。

系统还支持数据回传：`UART3_SendXY_Text(float PositionX, float PositionY)` 使用 `snprintf` 将浮点坐标格式化为字符串 `"%.6f,%.6f\n"`并通过 `HAL_UART_Transmit` 发送。虽然 BlueTooth.c 定义发送函数，但实际回传由 GoChassisTask 调用 `send_wheel_data(&data_)` 发送里程计估算的 `x_pos/y_pos`。

```c
void UART3_SendXY_Text(float PositionX, float PositionY)
{
    char buf[64];
    int n = snprintf(buf, sizeof(buf), "%.6f,%.6f\n", PositionX,
        PositionY);
    HAL_UART_Transmit(&huart3, (uint8_t*)buf, n, 100);
}
```

Code Listing 6: BlueTooth.c：坐标字符串回传（snprintf + HAL_UART_Transmit）

(4) 语音模块适配：ASR-PRO 十六进制指令 + UART 中断接收 语音模块通过 UART 串口通信实现控制：ASR-PRO 识别关键词后发送 3 字节十六进制数据包，STM32 串口中断接收并解析为指令 ID，写入全局变量 `Usart_Receive`，随后 FreeRTOS 任务执行动作。协议格式为包头 `0xA0` + 长度/类型 `0x01` + 指令 ID。STM32 与语音模块的串口波特率配置为 9600（`MX_USART3_UART_Init`），并通过 `HAL_UART_Receive_IT` 开启中断接收。当语音模块发送指令（如 04 打开刷子）时，STM32 解析指令 ID 并

映射到功能与运动控制：例如 Usart_Receive==4 调用 SetClearMotorSpeed(90,90)，Usart_Receive==5 开吸尘，Usart_Receive==6 开水泵，Usart_Receive==7/8 关闭对应设备；运动侧 Usart_Receive==1/2/3 分别触发前进/右转/左转等动作(如调用 forward_1()、PIDSpeedLoopRightFalse、PIDSpeedLoopLeftFalse)。

```
volatile uint8_t Usart_Receive = 0;
uint8_t Usart3_Receive_buf[3];

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART3){
        if(Usart3_Receive_buf[0] == 0xA0 && Usart3_Receive_buf[1] ==
           0x01){
            Usart_Receive = Usart3_Receive_buf[2]; // 指令ID
        }
        HAL_UART_Receive_IT(&huart3, Usart3_Receive_buf, sizeof(
           Usart3_Receive_buf));
    }
}
```

Code Listing 7: USART3：语音指令接收并写入 Usart_Receive（协议 0xA0 0x01 ID）

(5) 树莓派硬件适配：USART1 串口协议（Rx/Tx 双向） STM32 与树莓派之间的通信物理接口使用 USART1。系统定义两套数据帧格式：接收控制指令（树莓派→STM32）与回传状态数据（STM32→ 树莓派）。接收协议为定长 12 字节，帧头 0xAA、帧尾 0x55；发送协议为定长 16 字节，由 send_wheel_data 发送里程计或速度信息。接收处理采用“中断状态机 + 任务轮询”：在 HAL_UART_RxCpltCallback 中使用 frame_finding/frame_receiving 状态机寻找帧头并接收载荷，凑满 12 字节且帧尾校验正确后调用 process_frame；process_frame 使用 memcpy 将 4 字节数据解析为 float 并写入 left_speed_/right_speed_；在 SenfSerialTask 中将速度转换为整数目标（如 sspl=sspl*10），并根据左右轮速度相等与否决定优先直行（forward_1()）或差速控制（PIDSpeedLoop）。

状态回传由 GoChassisTask 周期性触发：任务通过 WheelData 结构体准备数据，其中 x_pos_t 由右轮速度 sp_r 乘以系数 0.3333（结合方向 flag_r）得到，y_pos_t 由左轮速度 sp_l 乘以 0.3333（结合 flag_l）得到，yaw_t 取反自全局航向角 yaw；随后在 send_wheel_data 中使用 memcpy 序列化并通过 HAL_UART_Transmit 发送 16 字节数据。发送周期由 osDelay(50) 控制，即 20Hz。

```
volatile float left_speed_ = 0.0f, right_speed_ = 0.0f;

```

```
void process_frame(uint8_t *buf)
{
    // buf: [0xAA ... payload ... 0x55]
    memcpy(&left_speed_,  &buf[1], 4);
    memcpy(&right_speed_, &buf[5], 4);
}
```

Code Listing 8: USART1：接收帧解析（memcpy 解出 left_speed_/right_speed_）

```
typedef struct{
    float x_pos_t;
    float y_pos_t;
    float yaw_t;
} WheelData;

void GoChassisTask(void const *argument)
{
    WheelData data_;
    for(;;){
        data_.x_pos_t = sp_r * 0.3333f;  // 结合flag_r可做方向处理
        data_.y_pos_t = sp_l * 0.3333f;  // 结合flag_l可做方向处理
        data_.yaw_t   = -yaw;

        send_wheel_data(&data_);
        osDelay(50); // 20Hz
    }
}
```

Code Listing 9: GoChassisTask：20Hz 回传结构体并发送（sp_r/sp_l * 0.3333, yaw 取反）

3.2 软件架构设计

软件架构采用分层设计：驱动层（HAL + CMSIS）提供对 RCC/GPIO/UART/-TIM/I2C 等外设的标准 API；底层支撑层（BSP）适配板级连接并封装协议细节，如位带操作宏（`PAout(n)`）、串口协议收发与帧校验（0xAA...0x55）、ADC 采样等；模块层（Module）以电机、传感器、编码器、蓝牙、IMU 等为独立模块，屏蔽业务逻辑；应用层（Application）承载运动学与控制逻辑，如 PIDSpeedLoop、机器人运动学分解与蓝牙指令映射；任务层（Task）以 FreeRTOS 组织任务生命周期与优先级，形成系统调度中心。

表 4: 分层架构与典型文件/职责

层级	典型文件	职责
Driver	STM32F4xx_HAL_Driver, CMSIS	外设与内核抽象；提供 HAL_UART_Transmit、HAL_TIM_PWM_Start 等统一 API。
BSP	bsp_sys.h, bsp_usart.c/h, bsp_adc.c/h, bsp_gpio.c/h	板级支持与协议封装；中断收发、帧头帧尾校验（0xAA...0x55）、ADC 电压监测等。
Module	Bluetooth.c/h, Sensors.c/h, MotorWheel.c/h, encoder.c/h, mpu9250	独立功能模块驱动，减少与业务逻辑耦合。
Application	robot.c/h, BlueToothControl.c/h, PIDController.c/h, ADCRead.c/h	运动学、控制算法、指令解析与业务逻辑实现。
Task	FreeRTOS: tasks.c, event_groups.c 等	多任务调度中心；划分 GimbalTask、SenfSerialTask、GoChassisTask 等。

3.3 核心技术实现

3.3.1 运动控制算法：串级 PID 角度式闭环控制

系统采用串级 PID 闭环以提升直线抗干扰与转向精度。外环为角度修正环：输入为目标航向角(如 `target_angle1/target_angle_t`)与 IMU 实测航向角(`yaw/current_angle`)之差，算法为位置式 PID，输出为差速修正量 `output1`，该值作为速度微调量叠加到基础速度上（在 `forward_1()`、`TurnToAngle_90()` 中体现）。内环为速度执行环：输入为左右轮目标速度（基础速度 ± 修正量），反馈为编码器速度 `sp_l/sp_r`，算法可为增量式或位置式 PID，输出最终 PWM 占空比 `pwm_l/pwm_r` 直接驱动电机。

3.3.2 数据采集：IMU 角度检测与编码器测速（mm/s）

IMU 角度检测使用 MPU9250 DMP 输出 yaw 并通过全局变量共享给控制任务。编码器测速中 `sp_l = cnt_l * 100` 可视为将 10ms 脉冲数换算到每秒脉冲数（PPS）。若要进一步换算为物理单位 mm/s，需要结合编码器线数、减速比与轮周长进行比例换算。原实现中在 GoChassisTask 回传数据时使用系数 0.3333 将脉冲速度转换为物理速度（`data_.x_pos_t = sp_r * 0.3333`），意味着 PID 参数整定是基于“脉冲数域”而

非严格物理单位域。

3.3.3 控制效果：直线抗干扰与转弯角度精准控制

转弯控制基于 IMU 的闭环而非简单“单轮停转”。系统设置目标角 `target_angle_t`，并包含过零处理逻辑(例如 > 180 或 < -180 的跳变处理)以适配 $-180°$ 到 $+180°$ 的航向角表示。原地旋转时左右轮速度大小相等方向相反(`leftsp=-output_t, rightsp=output_t`)，实现 Spot Turn。该策略能在地面摩擦变化、载荷变化等情况下持续修正，直到 yaw 角达到设定误差范围内才减速停止，从而提升角度控制精度。

3.3.4 通信协议开发：Web 端 3D 智能地面站（V4.7）

系统提供 Web 端机器人 3D 智能地面站（V4.7）用于浏览器直接控制与监控。该平台集成了数据可视化、蓝牙通信、路径规划与多模式控制：中心区域为 Canvas 绘制的交互式 3D 轨迹地图，实时显示机器人位置（X,Y）与航向（Yaw）并绘制历史轨迹；右侧仪表盘显示左右轮速度与航向角；左侧日志窗口打印系统状态、蓝牙连接信息与导航指令。控制方面支持键盘/触屏遥控（WASD、Q/E 功能键、Z/X 特殊指令）、自动导航(地图双击目标点触发规划)、区域覆盖（生成“弓字形”覆盖路径)、原路返回与直线回家等模式；同时提供 MATLAB Retro / Cyberpunk Neon / Clean Lab 三种 UI 主题切换。

图 4: 三种 UI 图)

Web Bluetooth 通信采用浏览器 `navigator.bluetooth` API 连接 BLE 设备，扫描服务 UUID 为 0xFFE0。下行指令通过 `sendByte(value)` 写入单字节控制码（例如 1 前进、2 后退、3 左转、4 右转、0 停止，5/6 对应 Q 键开关，7/8 对应 E 键开关，53/54 对应 Z/X 特殊指令)。上行数据通过 Notify 监听并解析定长 16 字节数据包：字节 0 为帧头 0x55，字节 2-5 为 Float32 左轮速度，字节 6-9 为 Float32 右轮速度，字节 10-13 为 Float32 航向角。解析使用 DataView 直接读取二进制，避免字符串解析开销。地图渲染采用等轴测投影 `project3D` 将 (x,y) 显示为伪 3D；里程计在 `updateOdometry` 中通过积分左右轮速度与 yaw 更新 `robotX/robotY`；交互支持拖拽平移与滚轮缩放。自动控制算法在前端 JS 实现，包含 Pure Pursuit 变种的路径跟随、引入终点锁定与动态前瞻距离以减少“S”形抖动；覆盖规划通过 `generateCoveragePath` 生成弓字形航点，并用射线法 `isPointInPoly` 判断区域内点。

```javascript
const SERVICE_UUID = 0xFFE0;
let characteristic = null;

async function linkToBlueTooth(){
  const device = await navigator.bluetooth.requestDevice({
    filters: [{ services: [SERVICE_UUID] }]
  });
  const server = await device.gatt.connect();
  const service = await server.getPrimaryService(SERVICE_UUID);
  characteristic = await service.getCharacteristic(0xFFE1);
}

async function sendByte(value){
  if(!characteristic) return;
  const data = new Uint8Array([value & 0xFF]);
  await characteristic.writeValue(data);
}
```

Code Listing 10: Web Bluetooth：服务 UUID=0xFFE0 连接与单字节指令发送（关键变量/函数）

4 上位机与系统协同实现

本章围绕 ROS 2 分布式架构下的上位机导航系统与整机协同运行机制展开，重点说明系统在规划、定位、导航、控制与可视化等模块之间的协同关系，以及其在实际扫地机器人平台上的工程落地方式。

4.1 ROS 2 系统协同架构

系统整体基于 ROS 2 框架构建，采用 DDS（Data Distribution Service）作为底层通信中间件。DDS 提供了高吞吐、低延迟的发布–订阅机制，使各功能模块以松耦合方式并行运行，适用于分布式机器人系统。

如图 5 所示，系统功能模块主要包括规划（Planning）、定位（Localization）、导航（Navigation）与控制（Control）等核心节点。各节点通过 DDS 数据总线进行通信，底层传感器数据（激光雷达、编码器、IMU 等）被统一发布，上层算法模块按需订阅，实现感知、决策与执行的解耦。

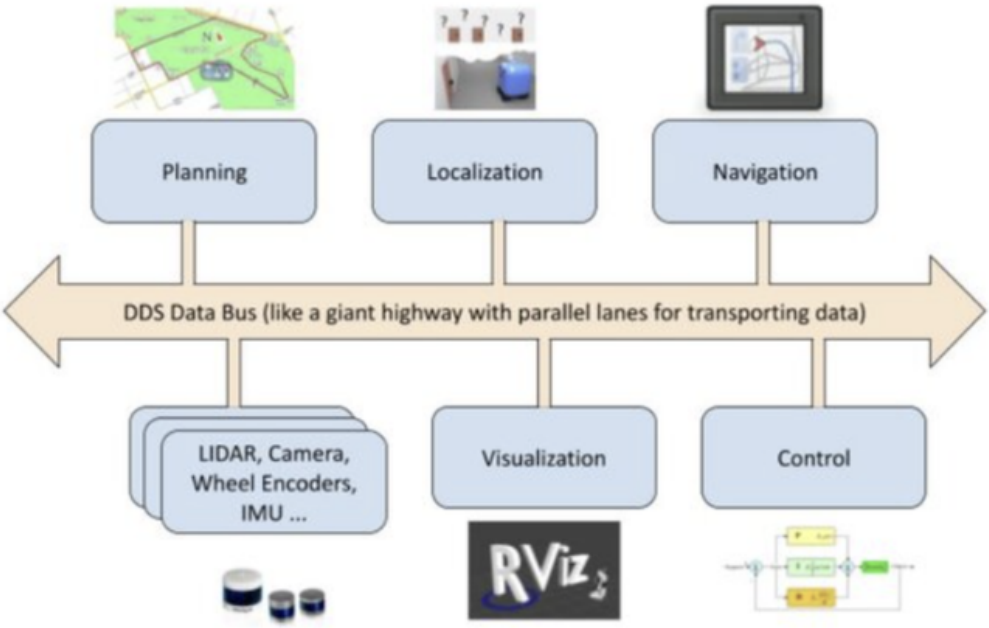


图 5: ROS 2 分布式系统架构与 DDS 数据总线示意图

在本系统中，STM32 负责底层实时控制；树莓派作为 ROS 2 边缘节点接入 DDS 网络；上位机作为核心计算节点，运行 SLAM、路径规划与可视化程序，三者协同完成完整的导航闭环。

4.2 ROS 2 软件工程组织与功能划分

为保证系统可维护性与功能扩展性，上位机与树莓派端的 ROS 2 工程采用模块化组织方式。整体工作空间为 `navigation_ws`，核心功能包为 `nav_system`，其结构与功能划分如表 5 所示。

表 5: ROS 2 导航系统工程结构与功能说明

目录 / 文件	功能说明
CMakeLists.txt	ROS 2 包编译规则配置文件
package.xml	功能包依赖声明与版本信息
config/slam.yaml	SLAM 算法相关参数配置
config/nav2.yaml	Nav2 导航栈参数配置
launch/sim.launch.py	仿真环境启动文件
launch/bringup.launch.py	实机系统一键启动文件
scripts/planner_node.py	路径规划节点实现
scripts/controller_node.py	运动控制与指令发布节点
map/	静态地图文件存放目录
rviz/	RViz 可视化配置文件
urdf/	机器人模型描述文件
worlds/	Gazebo 仿真环境模型

该工程结构将参数、启动逻辑、算法节点与可视化资源进行明确分离，便于在仿真与实机之间快速切换，同时支持独立调试与模块替换。

4.3 导航与路径规划实现

系统导航功能基于 ROS 2 Nav2 框架实现，支持在已构建地图中进行目标点导航、动态避障与路径重规划。导航流程为：定位节点提供机器人当前位姿，全局规划器生成路径，局部规划器结合代价地图输出实时控制指令。

4.3.1 RViz 导航交互与路径可视化

在 RViz 可视化界面中，系统可实时显示机器人在地图中的位置、全局路径与局部轨迹。操作者可直接在 RViz 中手动指定目标点，触发自主导航过程。

如图 6 所示，RViz 界面中清晰展示了地图轮廓、障碍物分布以及规划路径，便于对导航效果进行直观验证。

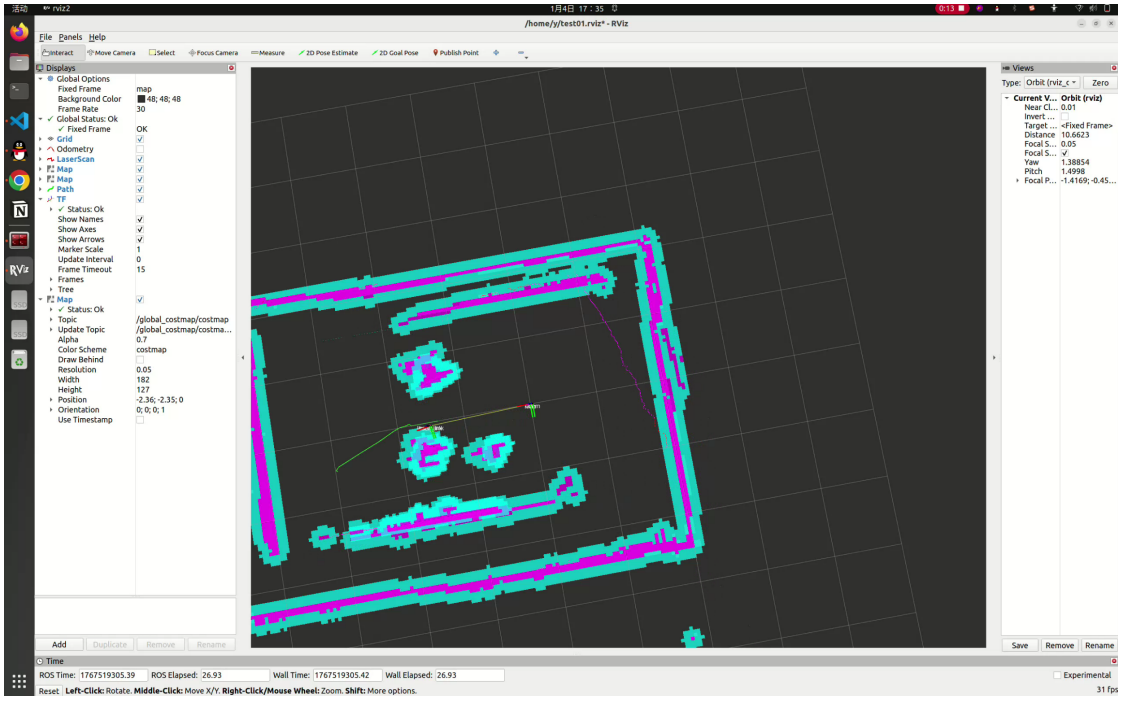


图 6: RViz 中的自主导航与路径规划效果展示

导航过程中，系统根据实时构建的代价地图动态调整路径，在保证安全距离的前提下完成目标点到达。全局路径与局部轨迹同时显示，有助于分析规划稳定性与避障行为。

4.3.2 关键导航参数设置

为适配扫地机器人尺寸与室内环境特性，系统对部分关键参数进行了针对性配置，主要包括机器人等效半径、安全边界与路径采样步长等。相关参数设置示例如下：

```
robot_radius_m = 0.25   // 机器人等效半径
margin_m       = 0.05   // 额外安全边界
step_m         = 0.40   // 覆盖采样步长
wp_spacing_m   = 0.50   // 相邻航点间距
```

Code Listing 11: 导航关键参数配置示例（安全距离与路径覆盖）

通过上述参数配置，系统在保证导航安全性的同时兼顾路径覆盖效率，适用于扫地机器人等室内移动平台。

4.4 导航执行状态判定与流程控制

在自主导航过程中，系统需要根据机器人当前位置与目标点之间的关系判断导航状态，以决定是否继续移动、减速或切换任务状态。以下给出一个核心状态判定函数示例。

```
def goal_reached(curr_x, curr_y, goal_x, goal_y, threshold):
    dx = curr_x - goal_x
```

```
    dy = curr_y - goal_y
    dist = (dx * dx + dy * dy) ** 0.5

    if dist < threshold:
        return True
    else:
        return False
```

Code Listing 12: 导航到达判定核心函数

当判定函数返回真时，系统将停止当前导航任务，并可触发下一步动作，如原地旋转、切换清扫区域或返回充电点。

4.5 系统协同流程总结

综合第四章所述设计，系统整体协同流程可总结为：

1. STM32 实时执行底盘控制与传感器采集；
2. 树莓派接收下位机和雷达数据并发布至 ROS 2 网络；
3. 上位机完成定位、路径规划与导航决策；
4. 控制指令经 ROS 2 网络下发至树莓派；
5. 树莓派转发指令至 STM32 执行；
6. 执行结果持续反馈，形成闭环控制。

5 功能验证与测试结果

为验证系统整体设计的正确性与稳定性，本项目围绕基础功能、核心功能与交互功能三个层面，对扫地机器人平台进行了系统性的功能测试。测试环境包括室内实际场景与 ROS 2 仿真环境，重点关注系统响应、执行准确性与安全可靠性。

5.1 基础功能测试

5.1.1 硬件功能测试

对机器人底层硬件功能进行逐项验证，包括电机启停、清扫刷、吸尘器、水泵运行以及蜂鸣器与 LED 指示灯显示等功能。测试结果表明，各执行机构均可根据控制指令正常启动与关闭，PWM 调速响应及时，运行状态与指示灯显示一致；蜂鸣器在报警与状态提示场景下可准确触发，满足基本交互需求。

5.1.2 数据采集功能测试

数据采集测试主要针对编码器速度数据与传感器数据采集准确性。编码器在不同转速条件下输出的速度变化趋势与实际运动状态一致，满足 PID 控制对反馈实时性的要求。红外传感器与碰撞光电门在接近障碍或触发条件下能够及时改变状态变量，数据采集稳定，无明显丢失或误判现象。

5.1.3 充电功能测试

充电功能测试包括充电触发、充电状态监测与电压转换稳定性验证。实验结果显示，机器人在对准充电座后可正常触发充电流程，充电状态能够被系统识别并反馈；电源模块在充电与非充电状态切换过程中工作稳定，电压转换未出现异常波动。

5.2 核心功能测试

5.2.1 运动控制测试

在直线行驶与转向场景下，对 PID 控制效果进行测试。实验表明，在存在轻微地面摩擦差异或外界干扰的情况下，机器人仍能保持较好的直线稳定性；在转弯测试中，基于 IMU 航向角闭环的控制策略可实现较高的角度精度，转向误差保持在可接受范围内，验证了串级 PID 控制策略的有效性。

5.2.2 安全防护测试

安全防护功能主要测试防跌落与碰撞检测响应能力。当悬空红外传感器检测到边缘状态时，系统能够及时触发停止或后退动作；在碰撞光电门触发情况下，机器人可快速响应并中断当前运动指令，整体安全防护机制可靠。

5.2.3 导航功能测试

导航功能测试在室内环境下进行，包括建图、自主导航、动态避障与回充导航等场景。系统可完成室内空间的 SLAM 建图，并在 RViz 中实时显示地图与位姿；在指定目标点后，机器人能够自主规划路径并成功到达目标位置；面对动态障碍物时，导航系统可进行路径调整；回充导航测试中，机器人能够在多数测试场景下成功返回充电区域，验证了导航系统的实用性。

5.3 交互功能测试

5.3.1 遥控功能测试

通过手机 APP 与键鼠方式对机器人进行遥控测试。结果显示，遥控指令响应及时，运动方向与操作输入一致，系统延迟处于可接受范围，满足人工干预与调试需求。

5.3.2 语音交互测试

语音交互测试主要验证语音指令识别与 AI 对话功能。实验中，语音模块可准确识别预设控制指令并触发对应动作；在 AI 对话场景下，系统交互流程连贯，整体体验流畅。

5.3.3 网页交互与数据可视化测试

网页端测试包括本地网页与 Github Page 页面加载、蓝牙数据传输稳定性以及数据可视化效果。测试结果表明，网页界面可正常加载并与机器人建立连接，蓝牙通信稳定；RViz 与网页端均可实时显示机器人状态数据，信息更新准确。

6 项目过程管理

6.1 项目进度

项目整体按照“需求分析 → 硬件设计 → 软件开发 → 算法部署 → 测试优化 → 结项”的流程推进。前期完成需求分析与方案论证，中期完成硬件原型制作与软件架构搭建，后期集中进行导航算法调试与系统联调，最终完成全功能验证并形成项目成果。

关键里程碑包括：硬件原型制作完成、嵌入式软件与 ROS 2 架构搭建完成、自主导航算法调试通过以及系统功能整体验证通过。

6.2 分工协作

项目采用分组协作方式推进：贺禄文负责电路设计、PCB 制作以及硬件焊接与调试；洪竞权负责嵌入式软件开发、驱动适配与通信协议实现；王令凯负责导航算法设计、ROS 2 系统部署与路径规划优化；共同负责系统功能测试、性能优化与问题排查，通过阶段性联调与沟通，确保系统整体进度与质量。

7 项目总结

7.1 核心创新点

本项目在多个层面体现了一定的创新性：在硬件方面，实现多模块高度集成与接口标准化设计，提升系统适配性；在软件方面，采用分层架构与多任务调度机制，增强系统稳定性与扩展能力；在算法方面，引入前瞻追踪与混合路径规划策略，兼顾导航平滑性与覆盖效率；在交互方面，融合遥控、语音与网页等多种交互方式，提升系统使用灵活性。

7.2 经验与教训

项目实施过程中积累了多方面经验：硬件与软件协同设计、模块化开发以及迭代式测试显著提高了开发效率。同时也暴露出一些不足，例如前期对模块兼容性考虑不充分、部分算法参数调试周期超出预期。

通过本项目进一步认识到跨领域协作的重要性，以及在项目初期明确需求和技术路线对整体进度与质量的关键作用。